

LIM M2

Processor Architecture Reference Manual

A deterministic, high-frequency 16-bit microprogrammed processor architecture featuring conflict-free multiplexer routing and 24-bit extended addressing.



POINT-TO-POINT DATA ROUTING · ZERO BUS CONTENTION

OFFICIAL REFERENCE SPECIFICATION

Revision 0.2 — October 2026

LIM M2 Core Architecture Manual. All rights reserved.

Contents

1	Document Scope and Architectural Philosophy	3
1.1	Notational Conventions	3
1.2	Unified Instruction Syntax Philosophy	3
2	Processor Overview	3
2.1	Conceptual Datapath Schematic	5
3	Architecturally Visible Registers	6
3.1	Flag and Status Register (FREG) Mapping	6
4	Machine Instruction Format	7
5	Instruction Set Architecture Summary	7
6	Detailed Instruction Specifications	8
	NOP — Hardware No-Operation and Pipeline Flush	9
	ADD — Addition with Automatic Multi-Word Carry Chaining	11
	SUB — Subtraction with Automatic Multi-Word Borrow Chaining	13
	MUL — Multiplication with Upper Word Residual Retention	15
	DIV — Division with Remainder Latched in ALU Residual	17
	LMR — Load External Data and Memory into Register	19
	LRM — Store Register Data into Memory and External Devices	21
	LRR — Register-to-Register Transfer (Copy from point B to point A)	23
	CMP — Hardware Operand Comparison	25
	JMP — Unconditional Program Branch	27
	JCR — Conditional Branch on Carry	28
	JNZ — Conditional Branch on Non-Zero	29
	JZ — Conditional Branch on Zero	30
	AND — Bitwise Logical AND	31
	OR — Bitwise Logical OR	32
	NAND — Bitwise Logical NAND	33
	NOR — Bitwise Logical NOR	34
	NOT — Bitwise Inversion (One's Complement)	35
	XOR — Bitwise Exclusive OR	36
	XNOR — Bitwise Equivalence	37
	INC — Atomic Increment by 1	38
	DEC — Atomic Decrement by 1	39
	PUSH — Push Word onto Execution Stack	40
	POP — Pop Word from Execution Stack	41

BSL — Barrel Shift Left	42
BSR — Barrel Shift Right	44
MSP — Manage Special Registers (FREG, PTREG)	46
RSV27 — Architectural Extension Slot	48
RSV28 — Architectural Extension Slot	49
RSV29 — Architectural Extension Slot	50
RSV30 — Architectural Extension Slot	51
RSV31 — Architectural Extension Slot	52
7 Physical Package and Pinout (34-Pin Form Factor)	53
7.1 Pin Descriptions	53
8 Interrupt Subsystem and Vector Processing	54
8.1 Unified 16-Vector Interrupt Space	54
8.2 Hardware Handshake Protocol (RDI → IA → D0–D7)	54
8.3 Context Preservation on 24-Bit Stack and Handler Entry	55
8.4 Return from Interrupt	55
9 Conclusion	55

1 Document Scope and Architectural Philosophy

This document serves as the formal architectural reference manual and hardware specification for the **LIM M2** microprocessor. The LIM M2 architecture has been deliberately designed to deliver deterministic execution, robust electrical integrity, and superior clock scaling.

A defining cornerstone of the LIM M2 datapath is the **total elimination of internal tri-state (high-impedance / Hi-Z) buses**. By substituting shared capacitive wires with actively driven multiplexer trees (Point-to-Point MUX Routing), the architecture prevents shoot-through currents, eliminates bus contention risks, and guarantees sharp, symmetrical transition edges.

Instruction Code Allocation

The LIM M2 instruction set architecture utilizes a contiguous 5-bit primary opcode space: NOP=0, ADD=1, ..., MSP=26. Opcode values 27–31 are reserved for next-generation hardware extensions.

1.1 Notational Conventions

Notation	Architectural Meaning
dst	Destination register targeted for writeback
src, srcA, srcB	Source register operand identifiers
imm	Immediate constant scalar embedded within the instruction
addr	Memory address (16-bit primary or 24-bit extended pointer)
cfg	3-bit operational mode configuration field
ZF, CF, OF	Hardware status flags (Zero, Carry, and Overflow) within FREG

1.2 Unified Instruction Syntax Philosophy

The LIM M2 processor architecture strictly enforces an orthogonal operand convention: **the destination operand (target, "where to") is always declared first, followed by input source operands ("where from")**. This ensures clean, deterministic code structure across all instructions:

- **Arithmetic Operations (3 Operands):** ADD Rd, Rs1, Rs2 denotes $Rd \leftarrow Rs1 + Rs2$ (e.g., ADD R0, R1, R2 computes $R0 = R1 + R2$); similarly for SUB, MUL, DIV.
- **Logic Operations (2 Operands):** AND Rd, Rs denotes $Rd \leftarrow Rd \text{ AND } Rs$ (e.g., AND R0, R1 computes $R0 = R0 \& R1$); similarly for OR, NAND, NOR, XOR, XNOR.
- **Bitwise Inversion NOT (1 Operand, 1-byte instruction):** NOT Rd denotes $Rd \leftarrow \sim Rd$ (e.g., NOT R0 computes $R0 = \sim R0$). Encoded in exactly 1 byte ([7:3] opcode, [2:0] Rd), no second byte is fetched from memory.
- **Register-to-Register Transfers:** LRR Rd, Rs executes a direct point-to-point copy from source (point B, Rs) to destination (point A, Rd): $Rd \leftarrow Rs$ (e.g., LRR R0, R1 computes $R0 = R1$).
- **Memory Loads (LMR):** the primary register argument Rd designates the processor register receiving loaded data.

2 Processor Overview

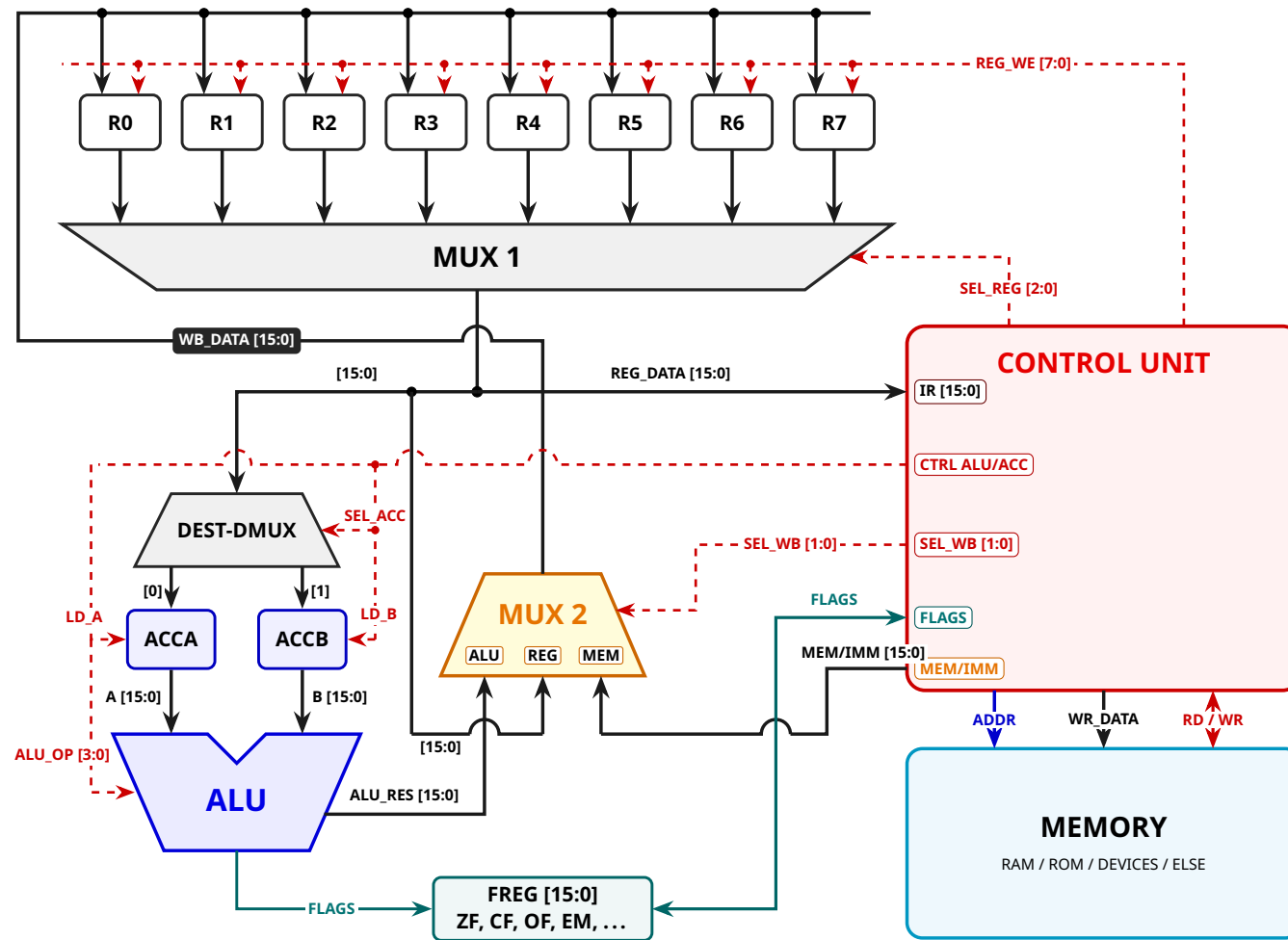
The **LIM M2** is a versatile, high-throughput 16-bit microprogrammed processor engineered for integer-intensive computation. Architectural highlights include:

- **Conflict-Free Multiplexer Datapath:** Internal signals are steered purely by push-pull multiplexer

trees (**MUX 1**, **DEST-DMUX**, and **MUX 2**). This modern design is perfectly suited for high-density ASIC and modern FPGA synthesis.

- **Dual Accumulator Staging:** Dedicated pre-ALU accumulators (ACCA and ACCB) isolate the arithmetic core from the general register file, minimizing propagation path skew.
- **Hardware ALU Auto-Chaining (Multi-Word Arithmetic):** The ALU integrates an internal residual register (ALU_RESIDUAL). When an addition overflows 16 bits ($> 2^{16} - 1$), a subtraction produces a borrow (< 0), or multiplication/division produces high-word residuals, the excess is retained in hardware and automatically accumulated or subtracted in the next mathematical operation. This enables continuous multi-word arithmetic (32, 48, 64+ bits) without dedicated ADC/SBB instructions. The chain is flushed using NOP.
- **Direct Reg-to-Reg Bypass:** A low-latency path from MUX 1 directly into MUX 2 enables single-cycle register transfers without locking ALU resources.
- **Linear 24-Bit Addressing (Up to 16 MB):** Dedicated 24-bit pointer registers (PTREG with auto-increment and SREG stack pointer) transcend the traditional 64 KB memory barrier, providing linear access across 16 megabytes of data and stack space.
- **High Clock Scaling Potential:** Unburdened by capacitive bus decay, the LIM M2 operates comfortably at a baseline frequency of **200 MHz**, scaling toward **700 MHz and above** in optimized silicon.

2.1 Conceptual Datapath Schematic



3 Architecturally Visible Registers

The programmer-visible register set of the LIM M2 processor comprises a clean, orthogonal hierarchy:

Register	Width	Architectural Purpose and Features
R0–R7	16-bit	General-Purpose Registers (GPR). Low-latency reading via MUX 1; atomic writeback from the WB_DATA highway under per-register write gating.
SREG	24-bit	Dedicated Stack Register. Supports efficient hardware frame and subroutine stack management with linear 24-bit stack addressing (up to 16 MB).
PTREG	24-bit	High-Precision Pointer Register. Provides flat linear addressing across 16 MB with optional hardware auto-increment.
FREG	16-bit	Status and Control Register. Unifies mathematical ALU condition flags with processor operational control bits.

3.1 Flag and Status Register (FREG) Mapping

The 16-bit FREG provides granular status monitoring and execution mode management:

Bit	Mnemonic	Functional Description
0	ZF	Zero Flag: Asserted when the execution result equals zero.
1	CF	Carry Flag: Captures arithmetic carry/borrow out of the most significant bit.
2	OF	Overflow Flag: Two's complement signed overflow indicator.
3	EM	Error Math: Hardware trap indicator for mathematical faults (e.g., divide-by-zero).
4–5	CMP	Compare Result: Hardware comparator status (00: reset/none/error, 01: less, 10: equal, 11: greater).
6–7	CLK	Clock Mode: Frequency prescaler and core clock domain configuration.
8	PIE	Program Interrupt Enable: Master mask for software interrupts.
9	MIE	Master Interrupt Enable: Hardware external interrupt enable.
10	DWR	Disable Wait for Ready: Forces immediate bus cycles ignoring slave wait states.
11	M24	24-bit Mode Enable: Activates flat 24-bit linear addressing.
12	AIP	Auto-Increment PTREG: Automatic hardware increment of PTREG upon memory access.
13	DBIT	Displaced Bit: Dedicated buffer for the bit ejected during shift operations.
14	SMOD	Shift Mode: Cyclic shift mode selector (0: zero-fill, 1: rotate through DBIT).
15	INTR	In Interrupt: Active interrupt execution flag (when 1, subsequent interrupts are locked out).

4 Machine Instruction Format

LIM M2 instructions follow an optimized, dense encoding format. A 5-bit primary opcode dispatches directly into the microcode sequencer:



The 3-bit cfg field selects up to 8 orthogonal execution modes per opcode, enabling expressive addressing and operand variations without opcode proliferation.

5 Instruction Set Architecture Summary

No.	Code	Mnemonic	Class	Functional Summary
0	00000	NOP	Control	No operation and ALU residual chain flush
1	00001	ADD	Arithmetic	Addition with automatic multi-word carry chaining
2	00010	SUB	Arithmetic	Subtraction with automatic multi-word borrow chaining
3	00011	MUL	Arithmetic	Integer multiplication with upper word residual latching
4	00100	DIV	Arithmetic	Integer division with remainder latched in ALU residual
5	00101	LMR	Memory	Load byte/word into register (src 0–3, size 8/16-bit)
6	00110	LRM	Memory	Store register byte/word into memory (dst 0–3, size 8/16-bit)
7	00111	LRR	Transfer	Low-latency register-to-register copy via bypass MUX
8	01000	CMP	Compare	Arithmetic compare without modifying operand registers
9	01001	JMP	Branch	Unconditional branch (direct, indirect, 24-bit pointer)
10	01010	JCR	Branch	Conditional branch on Carry Flag (CF)
11	01011	JNZ	Branch	Conditional branch on Non-Zero (ZF = 0)
12	01100	JZ	Branch	Conditional branch on Zero (ZF = 1)
13	01101	AND	Logic	Bitwise logical AND
14	01110	OR	Logic	Bitwise logical OR
15	01111	NAND	Logic	Bitwise logical NAND
16	10000	NOR	Logic	Bitwise logical NOR
17	10001	NOT	Logic	Bitwise bit inversion (one's complement)
18	10010	XOR	Logic	Bitwise exclusive OR
19	10011	XNOR	Logic	Bitwise equivalence
20	10100	INC	Arithmetic	Atomic increment operand by 1
21	10101	DEC	Arithmetic	Atomic decrement operand by 1
22	10110	PUSH	Stack	Push word onto execution stack
23	10111	POP	Stack	Pop word from execution stack into destination
24	11000	BSL	Shift	Barrel shift left with configurable fill
25	11001	BSR	Shift	Barrel shift right with discarded bit capture
26	11010	MSP	System	Manage special registers (FREG, PTREG)
27	11011	RSV27	Reserved	Reserved for vector/SIMD processing extensions
28	11100	RSV28	Reserved	Reserved for extended addressing modes

No.	Code	Mnemonic	Class	Functional Summary
29	11101	RSV29	Reserved	Reserved for floating-point coprocessor
30	11110	RSV30	Reserved	Reserved for hardware cryptographic acceleration
31	11111	RSV31	Reserved	Reserved for future architectural expansion

6 Detailed Instruction Specifications

The following pages detail the formal execution profiles for all 32 opcode allocations.

NOP

Hardware No-Operation and Pipeline Flush

00000
Control

Mnemonic	NOP
Opcode	00000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Control
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Hardware No-Operation and Pipeline Flush.

Architectural Semantics

Syntax: NOP
Operation: $PC \leftarrow PC + 1$, $ALU_RESIDUAL \leftarrow 0$, $FREG.CF \leftarrow 0$, $FREG.OF \leftarrow 0$
Example: NOP ; Flush ALU carry chain, align cycles

Configuration and Operands

No operands required; cfg field reserved for delay calibration.

Flags Affected (FREG)

Clears carry (CF), overflow (OF), and borrow tracking flags; preserves other flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Performs pipeline cycle alignment and flushes the internal ALU residual accumulator, terminating any chained multi-word carry/borrow sequence before starting new independent calculations.

TBD markers denote configurable hardware options.

ADD

Addition with Automatic Multi-Word Carry Chaining

00001
Arithmetic

Mnemonic	ADD
Opcode	00001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Addition with Automatic Multi-Word Carry Chaining.

Architectural Semantics

Syntax: `ADD arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 + arg2`
Operation: `sum <- arg1 + arg2 + ALU_RESIDUAL`
`arg0 <- sum[15:0], ALU_RESIDUAL <- (sum > 0xFFFF ? 1 : 0)`
Example: `ADD R0, R1, R2 ; R0 = R1 + R2 (registers R0-R7)`

Configuration and Operands

GPRs R0–R7, accumulators ACCA/ACCB, or immediate operands.

Flags Affected (FREG)

Sets carry flag CF and overflow OF if sum exceeds $2^{16} - 1$; updates ZF based on dst.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: [TBD](#).

Implementation Notes

Delivers seamless multi-word arithmetic (32, 48, 64+ bits). If the addition exceeds 16 bits ($> 2^{16} - 1$), the carry (+1) is latched into the internal ALU residual register and automatically added during

the subsequent ADD operation without requiring distinct ADC opcodes. The lower 16 bits are deterministically stored in dst. The chain is reset via NOP or overwritten by a new operation.

SUB

Subtraction with Automatic Multi-Word Borrow Chaining

00010
Arithmetic

Mnemonic	SUB
Opcode	00010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Subtraction with Automatic Multi-Word Borrow Chaining.

Architectural Semantics

Syntax: SUB arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 - arg2
Operation: diff <- arg1 - arg2 - ALU_RESIDUAL
arg0 <- diff[15:0], ALU_RESIDUAL <- (diff < 0 ? 1 : 0)
Example: SUB R0, R1, R2 ; R0 = R1 - R2 (registers R0-R7)

Configuration and Operands

GPRs R0-R7, constants, or accumulators ACCA/ACCB.

Flags Affected (FREG)

Sets borrow flag (CF) and OF on negative result (< 0); updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Enables continuous multi-word subtraction. When a borrow occurs (< 0), the borrow (−1) is retained in the internal ALU register and automatically subtracted during the next SUB operation. The lower

16-bit word is deterministically preserved in dst. Inserting a NOP breaks the chain for independent subtraction.

TBD markers denote configurable hardware options.

MUL

Multiplication with Upper Word Residual Retention

00011
Arithmetic

Mnemonic	MUL
Opcode	00011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Multiplication with Upper Word Residual Retention.

Architectural Semantics

Syntax: MUL arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 * arg2
Operation: prod <- arg1 * arg2
arg0 <- prod[15:0], ALU_RESIDUAL <- prod[31:16]
Example: MUL R0, R1, R2 ; R0 = R1 * R2 (upper 16 bits in ALU_RESIDUAL)

Configuration and Operands

GPRs R0–R7, accumulators ACCA/ACCB.

Flags Affected (FREG)

Updates ZF; sets OF/CF if the upper 16 bits of the product (prod[31:16]) are non-zero.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

For 32-bit products exceeding the 16-bit register width, the low 16 bits are placed into dst, while the upper 16 bits are retained in the internal ALU residual register. In a subsequent addition, this

residual is automatically accumulated (hardware MAC support / bigint multiplication). The upper word can also be extracted by adding the register to itself after clearing. Executing NOP flushes the residual register.

DIV

Division with Remainder Latched in ALU Residual

00100
Arithmetic

Mnemonic	DIV
Opcode	00100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Division with Remainder Latched in ALU Residual.

Architectural Semantics

Syntax: DIV arg0(REG), arg1(REG), arg2(REG) : arg0 = arg1 / arg2
Operation: arg0 <- arg1 / arg2
ALU_RESIDUAL <- arg1 % arg2
Example: DIV R0, R1, R2 ; R0 = R1 / R2 (remainder modulo in ALU_RESIDUAL)

Configuration and Operands

GPRs R0–R7, accumulators ACCA/ACCB.

Flags Affected (FREG)

Sets EM (Error Math) flag on division by zero without core stall; updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Stores the integer quotient in dst while preserving the remainder (modulo) in the internal ALU residual register for immediate reuse in subsequent operations or clearing via NOP.

TBD markers denote configurable hardware options.

LMR

Load External Data and Memory into Register

00101
Memory

Mnemonic	LMR
Opcode	00101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Memory
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Load External Data and Memory into Register.

Architectural Semantics

Syntax: LMR arg0(REG), <SRC_MODE> : arg0 = MEM[...]
Modes: SRC=0: LMR R0, [PC+] ; Imm Data (1-2 bytes)
SRC=1: LMR R0, [[PC+]] ; Imm Pointer
SRC=2: LMR R0, [PTREG] ; Pointer reg (AIP auto-inc)
SRC=3: LMR R0(R_hi), R1(R_lo) ; 24-bit composite address
Operation: arg0 <- MEM[EA] (upper address bits in R0 overwritten)
Example: LMR R0, R1 ; EA=(R0«16)|R1, load MEM[EA] into R0

Configuration and Operands

Octet 0: bit [8] (SZ) specifies operand width (0 = 1 byte, 1 = 2 bytes / word); bits [10 : 9] (SRC) select the data source (0 — immediate literal following instruction, 1 — immediate pointer following instruction, 2 — indirect via 24-bit pointer register PTREG, 3 — indirect via register pair).

Octet 1: bits [7 : 5] select register Arg1 (R_hi / destination), bits [4 : 2] select register Arg2 (R_lo). Under SRC = 3, the user specifies two 16-bit registers forming a 32-bit number, of which 24 bits are used in practice for physical addressing: Arg1 holds upper address bits and Arg2 holds lower 16 address bits. Notably, Arg1 also acts as the destination register: upon memory retrieval, the upper address bits in Arg1 are overwritten with the loaded data.

Flags Affected (FREG)

All flags in status register FREG are preserved without alteration.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.

2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: **TBD**.

Implementation Notes

The LMR instruction is the fundamental ingestion pathway bringing external data and memory into the LIM M2 general-purpose register file ($R0-R7$). The architecture unifies four orthogonal modes with native 24-bit addressing:

1. Under $SRC = 0$, immediate literals (1 or 2 bytes) are fetched directly from the instruction stream with automated PC advancement.
2. Under $SRC = 1$, an immediate pointer is read from code space, followed by data dereferencing.
3. Under $SRC = 2$, memory is addressed via the dedicated 24-bit pointer register PTREG. If the auto-increment flag $FREG.AIP$ (bit 12) is active, PTREG automatically advances by +1 or +2 in hardware, enabling single-cycle streaming array ingestion without software loop overhead.
4. Under $SRC = 3$, the user selects two 16-bit registers (Arg1 and Arg2) forming a 32-bit value, of which 24 bits are used for memory addressing. The user specifies that the upper address bits are placed in Arg1 and the lower 16 bits in Arg2. In this mode, register Arg1 (which held the upper address bits) also serves as the destination register: the retrieved data is saved into Arg1, overwriting its previous high-address value, while Arg2 continues to retain the lower 16 address bits.

TBD markers denote configurable hardware options.

LRM

Store Register Data into Memory and External Devices

00110
Memory

Mnemonic	LRM
Opcode	00110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Memory
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Store Register Data into Memory and External Devices.

Architectural Semantics

Syntax: LRM arg0(REG), <DST_MODE> : MEM[...] = arg0
Modes: DST=0: LRM R0, [PC+] ; Inline code memory
DST=1: LRM R0, [[PC+]] ; Imm Pointer
DST=2: LRM R0, [PTREG] ; Pointer reg (AIP auto-inc)
DST=3: LRM R0(R_hi), R1(R_lo) ; 24-bit composite address
Operation: MEM[EA] <- arg0[15:0] (or low byte if SZ=0)
Example: LRM R0, R1 ; EA=(R0«16)|R1, store R0 into MEM[EA]

Configuration and Operands

Octet 0: bit [8] (SZ) specifies operand width (0 = 1 byte, 1 = 2 bytes / word); bits [10 : 9] (DST) select the destination target (0 — inline memory directly following instruction, 1 — immediate pointer following instruction, 2 — indirect via 24-bit pointer register PTREG, 3 — indirect via register pair). **Octet 1:** bits [7 : 5] select source register Arg1 (Rs / R_hi), bits [4 : 2] select register Arg2 (R_lo). Under DST = 3, two registers form a 32-bit composite word of which 24 bits are used for physical addressing (Arg1 holds upper address bits and source data, Arg2 holds lower 16 bits). Overwriting existing data at the destination is the programmer's responsibility.

Flags Affected (FREG)

All flags in status register FREG are preserved without alteration.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).

3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

The LRM instruction is the fundamental egress pathway transferring register data from the LIM M2 general-purpose register file (*R0–R7*) to memory and memory-mapped I/O peripherals. The architecture unifies four orthogonal addressing modes:

1. Under $DST = 0$, the processor stores 1 or 2 bytes directly into memory following the instruction (PC) with automated PC advancement. This mode enables self-modifying routines and inline buffer writes; data destruction at the target address is the programmer's responsibility.
2. Under $DST = 1$, an immediate 16-bit pointer is read from code space, and data is stored to that dereferenced address.
3. Under $DST = 2$, memory is addressed via the dedicated 24-bit pointer register PTREG. If the auto-increment flag FREG.AIP (bit 12) is active, PTREG automatically advances by +1 or +2 in hardware, enabling single-cycle streaming array transfers without loop overhead.
4. Under $DST = 3$, two 16-bit registers are paired: Arg1 and Arg2 form a 32-bit composite integer, whose lower 24 bits specify the physical address (Addr[23 : 0]). Data from Arg1 is stored to the synthesized address, granting direct write access across the full 16 MB address space.

TBD markers denote configurable hardware options.

LRR

00111
Transfer

Register-to-Register Transfer (Copy from point B to point A)

Mnemonic	LRR
Opcode	00111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Transfer
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Register-to-Register Transfer (Copy from point B to point A).

Architectural Semantics

Syntax: LRR arg0(REG), arg1(REG) : arg0 = arg1
Operation: arg0 <- arg1 (direct copy from source to destination)
Example: LRR R0, R1 ; R0 = R1 (value of R1 copied to R0)

Configuration and Operands

Octet 0: opcode 00111₂ (0x07), bits [10 : 8] reserved (000₂).
Octet 1: bits [7 : 5] define destination register Arg1 (Rd, point A, receiving data); bits [4 : 2] define source register Arg2 (Rs, point B, providing data); bits [1 : 0] set to 00₂. When executed, Rs is copied to Rd in 1 microcycle.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

The LRR instruction provides the foundational register-to-register move mechanism across general-purpose registers ($R0-R7$). It executes a direct transfer from source (point B) to destination (point A) via the dedicated hardware **MUX 1** → **MUX 2** bypass datapath without stalling the ALU and without affecting FREG flags.

TBD markers denote configurable hardware options.

CMP

Hardware Operand Comparison

01000
Compare

Mnemonic	CMP
Opcode	01000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Compare
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Hardware Operand Comparison.

Architectural Semantics

Syntax: `CMP arg0(REG), arg1(REG) : compare(arg0, arg1)`
Operation: `if arg0 < arg1 then FREG.CMP <- 01b (less)`
`if arg0 == arg1 then FREG.CMP <- 10b (equal)`
`if arg0 > arg1 then FREG.CMP <- 11b (greater)`
(state 00b – initial reset / no comparison / error)
Example: `CMP R0, R1 ; 01: R0<R1, 10: R0==R1, 11: R0>R1 (00: reset/error)`

Configuration and Operands

Compares registers non-destructively.

Flags Affected (FREG)

Updates 2-bit FREG.CMP field (bits [5:4]) as well as ZF, CF, OF flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Enables swift condition evaluation without data corruption.

TBD markers denote configurable hardware options.

JMP

Unconditional Program Branch

01001

Branch

Mnemonic	JMP
Opcode	01001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Unconditional Program Branch.

Architectural Semantics

Syntax: `JMP target(IMM|REG) : PC = target`
Operation: `PC <- target`
Example: `JMP R0 ; Unconditional jump to address in R0`

Configuration and Operands

Supports 16-bit relative, direct absolute, or full 24-bit jump via PTREG.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Atomic reload of the Program Counter in minimal clock cycles.

TBD markers denote configurable hardware options.

JCR

Conditional Branch on Carry

01010
Branch

Mnemonic	JCR
Opcode	01010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Carry.

Architectural Semantics

Syntax: JCR target(IMM|REG) : if FREG.CF == 1 then PC = target
Operation: if FREG.CF == 1 then PC <- target
Example: JCR R2 ; Jump to address R2 if Carry Flag is set

Configuration and Operands

Target specified via relative displacement or absolute address.

Flags Affected (FREG)

Flags read without modification.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

Foundational primitive for multi-word extended precision arithmetic loops.

TBD markers denote configurable hardware options.

JNZ

Conditional Branch on Non-Zero

01011
Branch

Mnemonic	JNZ
Opcode	01011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Non-Zero.

Architectural Semantics

Syntax: JNZ target(IMM|REG) : if FREG.ZF == 0 then PC = target
Operation: if FREG.ZF == 0 then PC <- target
Example: JNZ R3 ; Jump to address R3 if ZF == 0 (non-zero)

Configuration and Operands

High-efficiency branch for loop counters and iterative structures.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Fast-path condition evaluation in microcode.

TBD markers denote configurable hardware options.

JZ

Conditional Branch on Zero

01100
Branch

Mnemonic	JZ
Opcode	01100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Branch
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Conditional Branch on Zero.

Architectural Semantics

Syntax: JZ target(IMM|REG) : if FREG.ZF == 1 then PC = target
Operation: if FREG.ZF == 1 then PC <- target
Example: JZ R4 ; Jump to address R4 if ZF == 1 (zero)

Configuration and Operands

Evaluates termination or equality conditions.

Flags Affected (FREG)

Preserves all status flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Optimized for string matching and search algorithms.

TBD markers denote configurable hardware options.

AND

Bitwise Logical AND

01101

Logic

Mnemonic	AND
Opcode	01101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical AND.

Architectural Semantics

Syntax: `AND arg0(REG), arg1(REG) : arg0 = arg0 & arg1`
Operation: `arg0 <- arg0 AND arg1`
Example: `AND R0, R1 ; R0 = R0 & R1`

Configuration and Operands

Register operands (2 operands).

Flags Affected (FREG)

Updates Zero Flag (ZF); resets overflow status.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Essential for bitmask extraction and masking.

TBD markers denote configurable hardware options.

OR

Bitwise Logical OR

01110
Logic

Mnemonic	OR
Opcode	01110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical OR.

Architectural Semantics

Syntax: OR arg0(REG), arg1(REG) : arg0 = arg0 | arg1
Operation: arg0 <- arg0 OR arg1
Example: OR R0, R1 ; R0 = R0 | R1

Configuration and Operands

Register operands (2 operands).

Flags Affected (FREG)

Updates ZF; carry/overflow deterministically cleared.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Rapid combination of control bitfields.

TBD markers denote configurable hardware options.

NAND

Bitwise Logical NAND

01111
Logic

Mnemonic	NAND
Opcode	01111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical NAND.

Architectural Semantics

Syntax: `NAND arg0(REG), arg1(REG) : arg0 = ~(arg0 & arg1)`
Operation: `arg0 <- NOT (arg0 AND arg1)`
Example: `NAND R0, R1 ; R0 = ~(R0 & R1)`

Configuration and Operands

Universal logic primitive (2 operands).

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Enables compact implementation of custom Boolean functions.

TBD markers denote configurable hardware options.

NOR

Bitwise Logical NOR

10000
Logic

Mnemonic	NOR
Opcode	10000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Logical NOR.

Architectural Semantics

Syntax: `NOR arg0(REG), arg1(REG) : arg0 = ~(arg0 | arg1)`
Operation: `arg0 <- NOT (arg0 OR arg1)`
Example: `NOR R0, R1 ; R0 = ~(R0 | R1)`

Configuration and Operands

Inverted logic primitive (2 operands).

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Broadens fast bitwise logic transformations.

TBD markers denote configurable hardware options.

NOT

Bitwise Inversion (One's Complement)

10001
Logic

Mnemonic	NOT
Opcode	10001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Inversion (One's Complement).

Architectural Semantics

Syntax: NOT arg0(REG) : arg0 = ~arg0
Operation: arg0 <- NOT arg0
Example: NOT R0 ; R0 = ~R0 (1-byte instruction)

Configuration and Operands

Single-byte instruction ([7:3] opcode 10001, [2:0] Rd). No second byte fetched from memory.

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Executes in minimal microcycles without fetching a second byte.

TBD markers denote configurable hardware options.

XOR

Bitwise Exclusive OR

10010
Logic

Mnemonic	XOR
Opcode	10010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Exclusive OR.

Architectural Semantics

Syntax: XOR arg0(REG), arg1(REG) : arg0 = arg0 ^ arg1
Operation: arg0 <- arg0 XOR arg1
Example: XOR R0, R1 ; R0 = R0 ^ R1 (XOR R0, R0 clears R0)

Configuration and Operands

Register operands (2 operands).

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Foundational for parity, CRC, and cryptographic ciphers.

TBD markers denote configurable hardware options.

XNOR

Bitwise Equivalence

10011
Logic

Mnemonic	XNOR
Opcode	10011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Logic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Bitwise Equivalence.

Architectural Semantics

Syntax: XNOR arg0(REG), arg1(REG) : $\text{arg0} = \sim(\text{arg0} \wedge \text{arg1})$
Operation: $\text{arg0} \leftarrow \text{NOT}(\text{arg0} \text{ XOR } \text{arg1})$
Example: XNOR R0, R1 ; $\text{R0} = \sim(\text{R0} \wedge \text{R1})$

Configuration and Operands

Bit-by-bit identity testing (2 operands).

Flags Affected (FREG)

Updates ZF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Fast parallel mask equivalence validation.

TBD markers denote configurable hardware options.

INC

Atomic Increment by 1

10100
Arithmetic

Mnemonic	INC
Opcode	10100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Atomic Increment by 1.

Architectural Semantics

Syntax: INC arg0(REG) : arg0 = arg0 + 1
Operation: arg0 <- arg0 + 1
Example: INC R0 ; R0 = R0 + 1 (increment register)

Configuration and Operands

Fast loop iterator step.

Flags Affected (FREG)

Updates ZF, OF; Carry Flag preserved or updated based on cfg.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Optimizes array index advancing.

TBD markers denote configurable hardware options.

DEC

Atomic Decrement by 1

10101
Arithmetic

Mnemonic	DEC
Opcode	10101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Arithmetic
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Atomic Decrement by 1.

Architectural Semantics

Syntax: DEC arg0(REG) : arg0 = arg0 - 1
Operation: arg0 <- arg0 - 1
Example: DEC R0 ; R0 = R0 - 1 (decrement register)

Configuration and Operands

Fast downward counter step.

Flags Affected (FREG)

Updates ZF, OF.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Compact loop termination construct.

TBD markers denote configurable hardware options.

PUSH

Push Word onto Execution Stack

10110
Stack

Mnemonic	PUSH
Opcode	10110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Stack
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Push Word onto Execution Stack.

Architectural Semantics

Syntax: PUSH arg0(REG) : SREG -= 2, MEM[SREG] = arg0
Operation: SREG <- SREG - 2, MEM[SREG] <- arg0
Example: PUSH R0 ; Push word R0 onto execution stack

Configuration and Operands

Source: any GPR (R0–R7) or special status register.

Flags Affected (FREG)

Preserves arithmetic flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Hardware-assisted subroutine and context preservation.

TBD markers denote configurable hardware options.

POP

Pop Word from Execution Stack

10111
Stack

Mnemonic	POP
Opcode	10111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Stack
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Pop Word from Execution Stack.

Architectural Semantics

Syntax: POP arg0(REG) : arg0 = MEM[SREG], SREG += 2
Operation: arg0 <- MEM[SREG], SREG <- SREG + 2
Example: POP R0 ; Pop word from top of stack into R0

Configuration and Operands

Destination: any GPR or status register.

Flags Affected (FREG)

Preserves arithmetic flags.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Symmetric restoration of execution contexts.

TBD markers denote configurable hardware options.

BSL

Barrel Shift Left

11000
Shift

Mnemonic	BSL
Opcode	11000 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Shift
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Barrel Shift Left.

Architectural Semantics

Syntax: BSL arg0(REG), arg1(REG), arg2(IMM|REG) : arg0 = arg1 « arg2
Operation: arg0 <- arg1 « arg2
FREG.DBIT <- ejected MSB (bit 15)
When SMOD=1: inject previous DBIT into LSB; when SMOD=0: 0
Example: BSL R0, R1, 1 ; Shift R1 left by 1 bit, store in R0

Configuration and Operands

Shift count: immediate constant or dynamic register value.

Flags Affected (FREG)

Ejected MSB latched into FREG.DBIT (bit 13). When SMOD=1, former DBIT is injected into LSB; when SMOD=0, vacant bits are zero-filled.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Streamlined multi-word rotation and bitfield packing.

TBD markers denote configurable hardware options.

BSR

Barrel Shift Right

11001
Shift

Mnemonic	BSR
Opcode	11001 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Shift
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Barrel Shift Right.

Architectural Semantics

Syntax: BSR arg0(REG), arg1(REG), arg2(IMM|REG) : arg0 = arg1 » arg2
Operation: arg0 <- arg1 » arg2
FREG.DBIT <- ejected LSB (bit 0)
When SMOD=1: inject previous DBIT into MSB; when SMOD=0: 0
Example: BSR R0, R1, 1 ; Shift R1 right by 1 bit, store in R0

Configuration and Operands

Logical or sign-extended arithmetic right shift selectable via cfg.

Flags Affected (FREG)

Ejected LSB latched into DBIT (bit 13). When SMOD=1, former DBIT is injected into MSB; when SMOD=0, vacant bits are zero-filled.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
2. Operation dispatch to dedicated execution block (ALU / memory / control).
3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
4. Synchronous status update of FREG and pointer registers (PTREG, SREG).

Cycle breakdown and control strobes: TBD.

Implementation Notes

Fast streaming scaling and power-of-two division.

TBD markers denote configurable hardware options.

MSP

Manage Special Registers (FREG, PTREG)

11010
System

Mnemonic	MSP
Opcode	11010 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	System
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Manage Special Registers (FREG, PTREG).

Architectural Semantics

Syntax: MSP sreg, [op], args : special register operations
Operation: **Direct Load (Bit 10 = 0):**
sreg=00 (FREG): FREG <- Rs[15:0] (Octet 1: [7:5]=Rs)
sreg=01 (PTREG): PTREG <- (Rs_hi << 16) | Rs_lo (Octet 1: [7:5]=Rs_hi, [4:2]=Rs_lo)
Modify/Operate (Bit 10 = 1):
sreg=00 (FREG): FREG <- FREG $\odot$ Rs (Octet 1: [7:6]=op {AND,OR,XOR,XNOR}, [5:3]=Rs)
sreg=01 (PTREG): PTREG <- PTREG $\pm$ Rs (Octet 1: [7]=op {ADD,SUB}, [6:4]=Rs)
Example: MSP FREG, R0 ; Direct load FREG = R0
MSP PTREG, R0, R1 ; Load 24-bit PTREG = {R0, R1}
MSP FREG, OR, R2 ; Apply bitmask: FREG = FREG | R2
MSP PTREG, ADD, R3 ; Pointer displacement: PTREG = PTREG + R3

Configuration and Operands

Special registers FREG (00) and 24-bit PTREG (01). Supports direct initialization, bitmask manipulation for FREG, and local pointer arithmetic for PTREG.

Flags Affected (FREG)

FREG flags modified directly or by mask. PTREG arithmetic updates CF carry/borrow via the ALU.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.

2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: **TBD**.

Implementation Notes

Universal privileged control primitive for system architectures.

RSV27

Architectural Extension Slot

11011
Reserved

Mnemonic	RSV27
Opcode	11011 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

Syntax: RSV27 : reserved
Operation: Reserved for LIM architecture hardware extensions

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV28

Architectural Extension Slot

11100
Reserved

Mnemonic	RSV28
Opcode	11100 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

Syntax: RSV28 : reserved
Operation: Reserved for LIM architecture hardware extensions

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV29

Architectural Extension Slot

11101
Reserved

Mnemonic	RSV29
Opcode	11101 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

Syntax: RSV29 : reserved
Operation: Reserved for LIM architecture hardware extensions

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV30

Architectural Extension Slot

11110
Reserved

Mnemonic	RSV30
Opcode	11110 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

Syntax: RSV30 : reserved
Operation: Reserved for LIM architecture hardware extensions

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

RSV31

Architectural Extension Slot

11111
Reserved

Mnemonic	RSV31
Opcode	11111 (Upper 5 bits of instruction word)
Encoding	5-bit opcode, 3-bit cfg field, followed by dynamic arguments (direct arguments for specialized commands)
Class	Reserved
Status	Core specification ratified; argument bitfields and extended options undergoing final layout

Instruction Purpose

Architectural Extension Slot.

Architectural Semantics

Syntax: RSV31 : reserved
Operation: Reserved for LIM architecture hardware extensions

Configuration and Operands

Reserved for future LIM ISA revisions.

Flags Affected (FREG)

Undefined.

Datapath Micro-Operations

1. Multiplexer selection of source operands from register file or immediate bus.
 2. Operation dispatch to dedicated execution block (ALU / memory / control).
 3. Result routing via Writeback selector (MUX 2) to destination register with write-enable gating.
 4. Synchronous status update of FREG and pointer registers (PTREG, SREG).
- Cycle breakdown and control strobes: TBD.

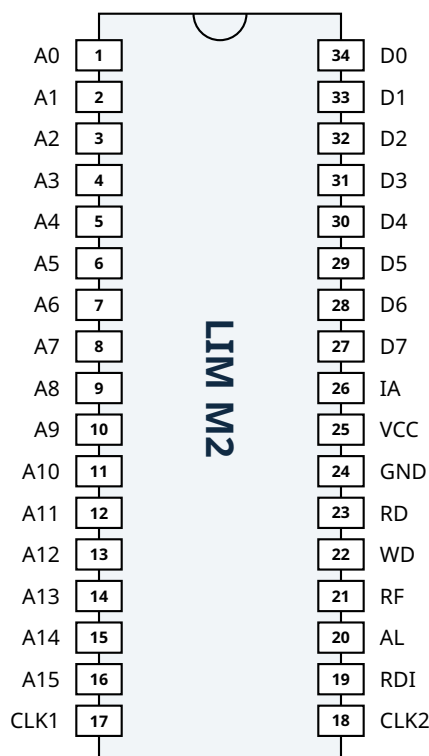
Implementation Notes

Reserved.

TBD markers denote configurable hardware options.

7 Physical Package and Pinout (34-Pin Form Factor)

The processor is packaged in a high-reliability dual-inline 34-pin enclosure:



7.1 Pin Descriptions

Signal	Direction	Functional Role
A0–A15	Output / Tri-State	16-bit primary address bus (latched via external AL strobe).
D0–D7	Bidirectional	8-bit external memory and peripheral data interface.
CLK1, CLK2	Input	Two-phase non-overlapping clock synchronization inputs.
VCC, GND	Power	Primary power supply rails (+5 V nominal).
RD, WD	Output	Bus Read Enable and Write Enable strobes.
AL	Output	Address Latch strobe for external high-speed address registers.
RF	Input	Ready Flag: external slave memory handshake acknowledge.
RDI	Input	Raw Direct Interrupt: priority hardware interrupt request line.
IA	Output	Interruption Accepted / Interrupt Acknowledge: informs peripherals of accepted request; core then samples interrupt vector from data bus.

8 Interrupt Subsystem and Vector Processing

The LIM M2 interrupt subsystem is architected to guarantee deterministic low-latency real-time response, bus contention immunity, and atomic context switching. The architecture integrates external hardware service requests and internal software traps into a unified 16-vector hierarchy governed by a cycle-accurate hardware handshake.

8.1 Unified 16-Vector Interrupt Space

The processor provides exactly 16 system interrupt vectors (indices 0–15), which are orthogonally shared between peripheral devices and internal CPU events:

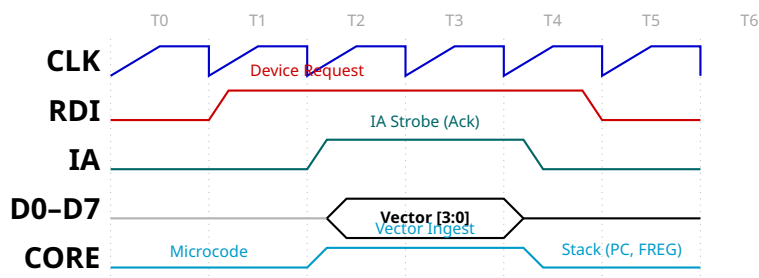
Table 3: LIM M2 Unified 16-Vector Interrupt Table

Vector	Code	Source / Type	Functional Role
VEC 0	0x00	Hardware / Reset	Power-on Reset: primary core and register initialization.
VEC 1	0x01	Hardware (NMI)	Non-Maskable Interrupt: power-fail warning, critical hardware alert.
VEC 2	0x02	Software Trap	Arithmetic Exception (division by zero, FREG.EM flag).
VEC 3	0x03	Software Trap	Operating System System Call (Syscall / API trap dispatcher).
VEC 4	0x04	Hardware / Periph	System Interval Timer / High-resolution scheduler tick.
VEC 5	0x05	Hardware / Periph	Serial Communications Interface (UART RX/TX service).
VEC 6	0x06	Hardware / Periph	Direct Memory Access Controller (DMA Transfer Complete).
VEC 7	0x07	Hardware / Periph	External Interrupt Controller / Expansion bus bridge.
VEC 8–15	0x08–0x0F	Shared (HW / SW)	User hardware interrupts and software-defined handlers.

8.2 Hardware Handshake Protocol (RDI → IA → D0–D7)

For robust peripheral interfacing, the processor implements a 4-phase hardware handshake cycle that completely eliminates CPU polling overhead:

- Request Phase (RDI):** A peripheral device asserts an active-high level on the RDI (*Raw Direct Interrupt*) pin. The core samples RDI at atomic micro-op boundaries.
- Arbitration Phase (Mask Check):** If interrupts are enabled (FREG.MIE = 1), the CPU commits the current datapath micro-operation and initiates the interrupt sequence.
- Acknowledge Phase (IA Strobe):** The core asserts an active-high strobe on the IA (*Interrupt Accepted / Interrupt Acknowledge*, Pin 26) output. This informs all slave devices on the bus that the core is ready to ingest the interrupt vector index.
- Vector Transfer Phase (D0–D7):** Concurrently with asserting IA, the processor switches the external bidirectional data bus D0–D7 to input mode and activates the RD line. The requesting peripheral drives its 8-bit vector ID onto D0–D7 (lower 4 bits address vectors 0–15).
- Latch and Dispatch Phase:** The CPU latches the vector ID from D0–D7, deasserts IA, and advances to context preservation.



8.3 Context Preservation on 24-Bit Stack and Handler Entry

Interrupt service entry is permitted strictly when the CPU is not already executing an interrupt handler ($\text{FREG.INTR} = 0$) and master interrupts are unmasked ($\text{FREG.MIE} = 1$). When $\text{FREG.INTR} = 1$, subsequent hardware interrupt requests are locked out by hardware.

Upon interrupt acceptance, the core executes an atomic state preservation sequence:

1. **Program Counter Preservation (PC):** The current instruction pointer is pushed onto the 24-bit execution stack via SREG:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{PC}_{15:0}$$

2. **Status and Flags Preservation (FREG):** The complete status register (with $\text{FREG.INTR}=0$) is pushed onto the stack:

$$\text{SREG} \leftarrow \text{SREG} - 2, \quad \text{Memory}[\text{SREG}] \leftarrow \text{FREG}$$

3. **Nested Interrupt Lockout:** The core asserts the in-interrupt flag $\text{FREG.INTR} \leftarrow 1$ (bit 15) and clears the mask bit $\text{FREG.MIE} \leftarrow 0$, precluding nested re-entrancy and race conditions.
4. **Vector Dispatch:** The handler address is indexed from the vector table using the ingested vector ID and loaded into PC.

8.4 Return from Interrupt

Exiting the interrupt service routine restores the preempted execution context:

1. The saved flags and processor modes are restored from the stack:

$$\text{FREG} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

(Restoring FREG automatically clears FREG.INTR back to 0, lifting the lockout, and restores FREG.MIE to its previous state).

2. The return address is popped back into the program counter:

$$\text{PC} \leftarrow \text{Memory}[\text{SREG}], \quad \text{SREG} \leftarrow \text{SREG} + 2$$

3. Normal instruction execution seamlessly resumes on the subsequent clock cycle with zero data corruption.

9 Conclusion

The **LIM M2** architecture harmonizes the time-tested reliability of deterministic microprogramming with the modern performance of an entirely tri-state-free multiplexer datapath. By eliminating bus contention, floating nodes, and capacitive decay, the LIM M2 guarantees exceptional signal fidelity and opens an unprecedented clock frequency envelope spanning from 200 MHz to the highest thresholds of modern silicon.